

Report on miniCPU Project

Mingxuan Li [2025533130] Ruiyang Xu [2025531125] Kaiwen Yang [2025531126]

1 Task Description

miniCPU Design and Implementation is a cutting-edge educational initiative focused on computer architecture.

In this project, our work revolves around building the datapath based on `Logisim`, developing an assembler in `Python`, and writing test cases.

2 Team's Timeline

2.1 Week 1

In week 1, we gained preliminary understanding of the concept of electrical level and sequential logic, and have studied the structure of registers. We also preliminarily developed a Python-based assembler.

2.2 Week 2

In week 2, we studied the fundamentals of computer architecture, including learning RISC-V instruction set and gaining an understanding of the datapath of a single-cycle CPU. Simultaneously, we refined our assembler, enabling it to read assembly files in the terminal, convert assembly instructions into hexadecimal machine code, and output the results via a text file.

2.3 Week 3

In week 3, we implemented a single-cycle CPU in Logisim capable of executing 13 RV32I instructions: `add`, `addi`, `sub`, `and`, `andi`, `or`, `ori`, `slt`, `slti`, `lw`, `sw`, `bne` and `jal`.

2.4 Week 4

In week 4, we designed test cases and conducted debugging and modifications on the CPU.

3 CPU Implementation

3.1 Overall Design Plan

- Instructions Set

R-Type	I1-Type	S-Type	J-Type	B-Type
<code>add</code> , <code>sub</code> , <code>or</code> , <code>and</code> , <code>slt</code>	<code>addi</code> , <code>andi</code> , <code>ori</code> , <code>lw</code> , <code>slti</code>	<code>sw</code>	<code>jal</code>	<code>bne</code>

• Modules Design

- *Instruction Fetch*: IFU includes the PC, NPC, and IM, where the PC register has an asynchronous reset function with a starting address of 0x00000000.
- *Instruction Memory*: IM is a read-only memory module that stores the machine code instructions in hexadecimal format generated by the assembler.
- *Register File*: RF contains 32 registers, each 32 bits wide. It supports two asynchronous read ports and one synchronous write port, and has an asynchronous reset function that initializes all registers to 0.
- *ALU*: ALU performs arithmetic and logical operations, supporting operations such as addition, subtraction, bitwise AND, bitwise OR, and set less than.
- *Data Memory*: DM is a synchronous read-write memory module. It has an asynchronous reset function that initializes all memory locations to 0.
- *Extend*: The extend module is responsible for extension of immediate values.
- *Control Unit*: The control unit generates control signals for each module and data path based on the opcode and funct3/funct7 fields of the instruction.

• Overall Structure

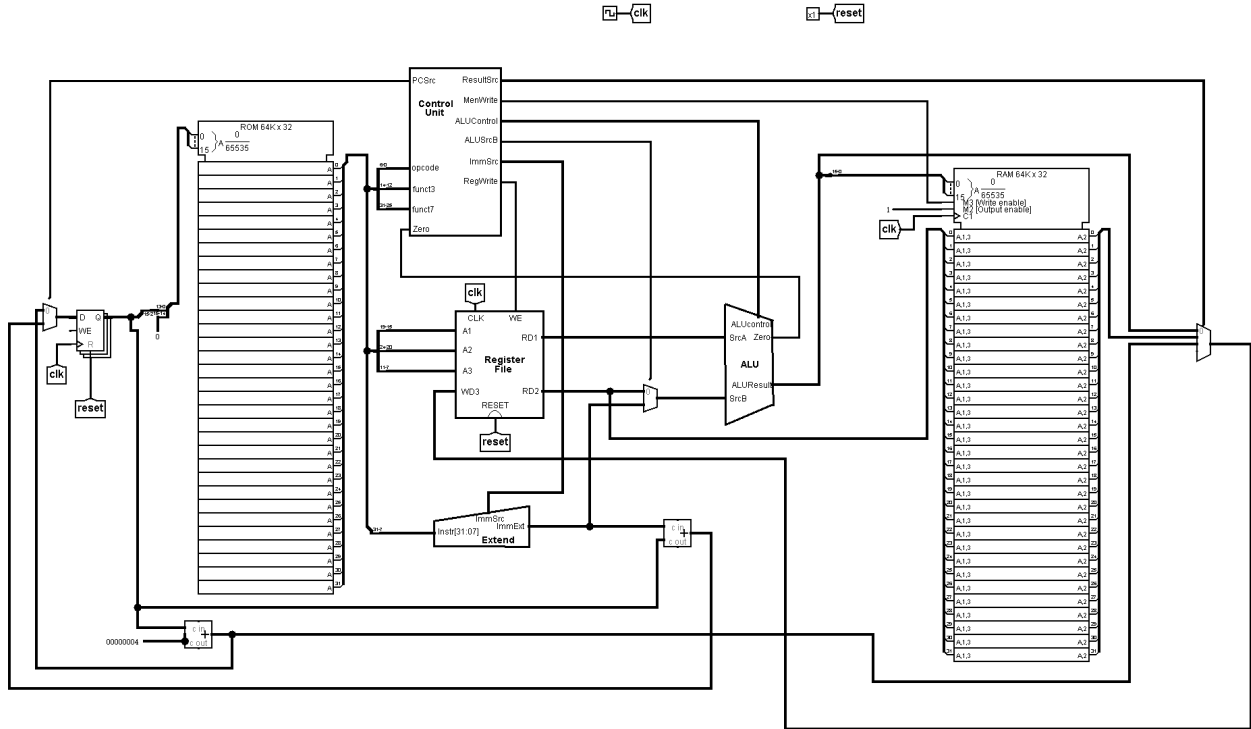


Figure 1: The overall structure

• Key Modules

1. PC

Port	Description
clk	Receives the clock signal (rising edge triggered).
reset	Receives an asynchronous reset signal, which sets the current instruction to the starting position when it is 1'b1.
D	Receives the next instruction address.
Q	Outputs the current instruction address.

Table 1: PC Module Ports

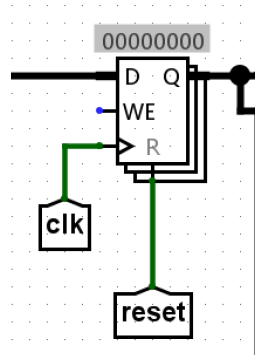


Figure 2: PC Module Implementation

2. Instruction Memory

We use a ROM as the instruction memory.

The physical address space of the instruction memory (i.e., the actual address range connected to the ROM) is 0x0000 to 0xFFFF (a total of 64KB of space). Since the Program Counter (PC) is 32 bits wide, but the instruction ROM only supports a 16-bit address input (a 32-bit address would require excessively large memory and is unnecessary), we only use the lower 16 bits of the PC (PC[15:0]) as the access address for the ROM. Each instruction is 32 bits wide (4 bytes), but the memory is word-addressable (32 bytes per word), where each address location stores 32 bits. Therefore, the address of the next instruction in this ROM is the current address plus 1.

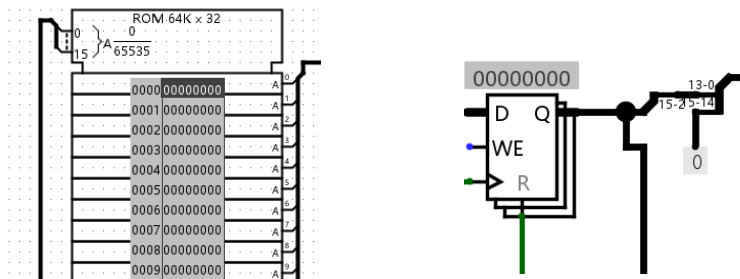


Figure 3: Instruction Memory Addressing Method

3. Register File

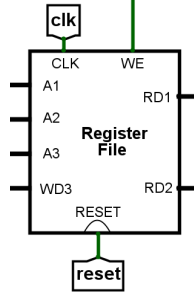


Figure 4: Register File Overall Structure

Port	Description
A1	Address of the first read register.
A2	Address of the second read register.
A3	Address of the write register.
WD	Data to be written to the write register.
RD1	Data output from the first read register.
RD2	Data output from the second read register.
WE	Write enable signal. When 1'b1, data is written to the register A3.
clk	Receives the clock signal (rising edge triggered).
reset	Receives an asynchronous reset signal.

Table 2: Register File Module Ports

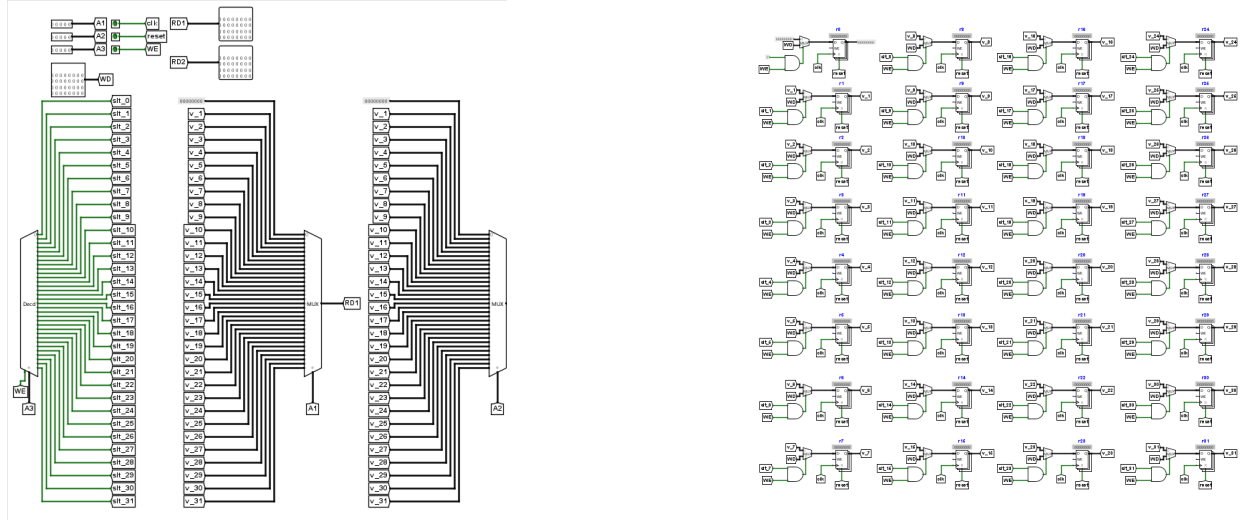


Figure 5: Register File Internal Structure

The specific implementation idea of a Register File is relatively straightforward. The `s1t` signal, obtained after decoding the `A3` input signal via a Decoder, represents the

selection signal for each individual register cell. v denotes the value stored in each register cell. The $A1$ and $A2$ input signals are routed through Multiplexers (MUX) to produce $RD1$ and $RD2$, respectively.

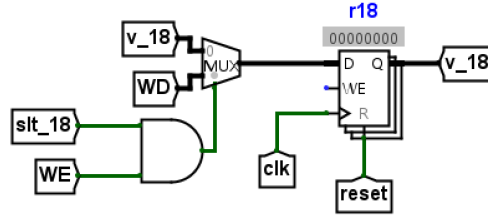


Figure 6: Register Cell Internal Structure

For each individual register cell:

clk receives the clock signal, and **reset** receives an asynchronous reset signal.

A **MUX** selects the content to be written into the register on the rising edge.

The **write data (WD)** will be written into the register only if both the **select signal (slt)** and the **global write enable (WE)** are at a high level. Otherwise, the existing value will be rewritten.

4. ALU

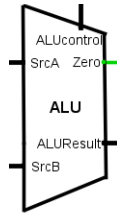


Figure 7: ALU Module Overall Structure

Port	Description
SrcA	First operand input.
SrcB	Second operand input.
ALUcontrol	Receives ALU operation code. 3'b000 A + B 3'b001 A - B 3'b010 A & B 3'b011 A B 3'b100 A < B
ALUResult	ALU output.
Zero	Zero flag indicating if the result is zero. Provide the basis for the judgment of the bne conditional jump

Table 3: ALU Module Ports

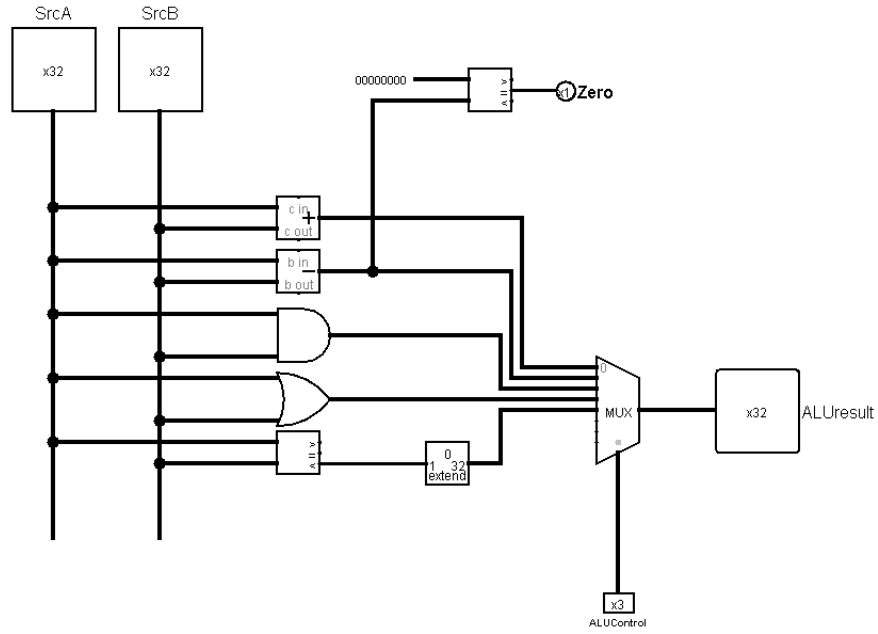


Figure 8: ALU Module Internal Structure

5. *Extend*

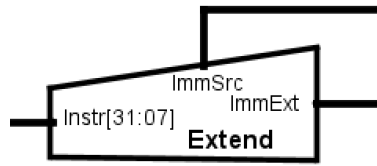


Figure 9: Extend Module Overall Structure

Port	Description
Instr [31 : 07]	Remove the opcode from the machine code and extract the immediate part
ImmSrc	Receives the immediate type code. 2'b00 I1-type 2'b01 S-type 2'b10 B-type 2'b11 J-type
ImmExt	Output the extended immediate value.

Table 4: Extend Module Ports

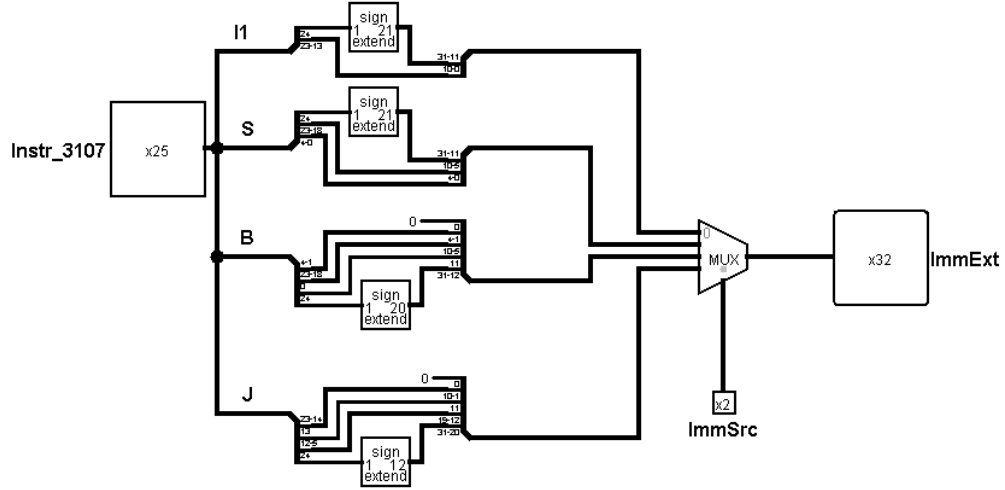


Figure 10: Extend Module Internal Structure

Immediate Type	Extraction Method
I1-type	20Instr[31],Instr[31 : 20]
S-type	20Instr[31],Instr[31 : 25],Instr[11 : 7]
B-type	20Instr[31],Instr[07],Instr[30 : 25],Instr[11 : 08],1'b0
J-type	12Instr[31],Instr[19 : 12],Instr[20],Instr[30 : 21],1'b0

Table 5: Immediate Extraction Methods

6. Data Memory

The idea is also relatively simple; just use a RAM.

The address width is 16 bits, and the data width is 32 bits.

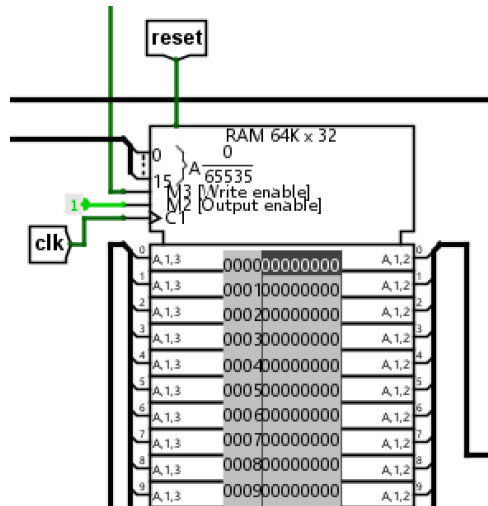


Figure 11: Data Memory Module Implementation

7. Control Unit

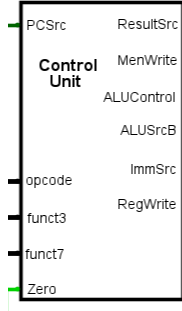


Figure 12: Control Unit Overall Structure

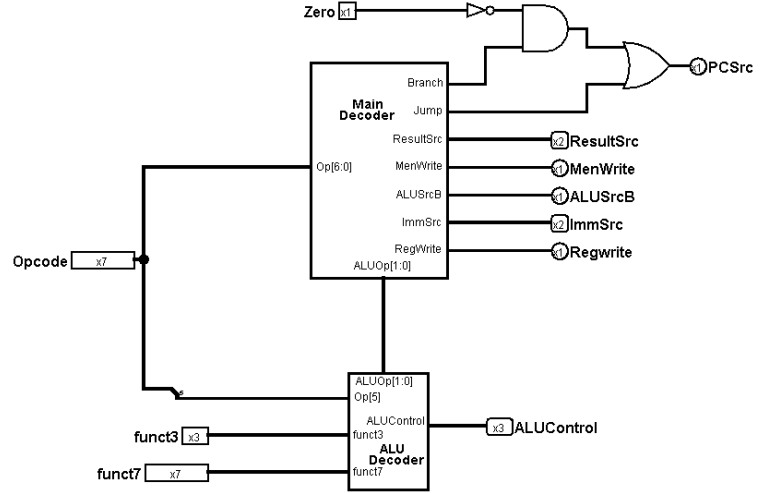


Figure 13: Control Unit Secondary Structure

Figure 14: Control Unit Structure

Port	Description
opcode	Receives the opcode field from the instruction.
funct3	Receives the funct3 field from the instruction.
funct7	Receives the funct7 field from the instruction.
ALUOp	Output ALU decoder control signal.
PCSrc	Control signal for PC MUX.
ResultSrc	Control signal for Result MUX.
MenWrite	Data Memory write enable signal.
ALUControl	ALU operation control signal.
ALUSrcB	ALU second operand MUX control signal.
ImmSrc	Immediate extension type control signal.
RegWrite	Register File write enable signal.

Table 6: Control Unit Module Ports

The Control Unit is divided into the **Main Decoder** and the **ALU Decoder**.

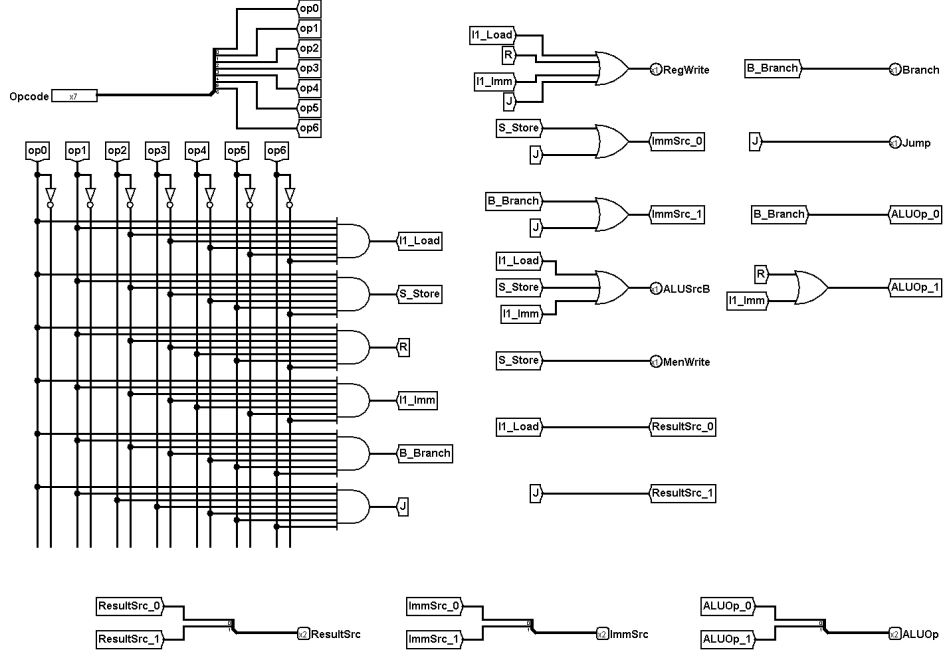


Figure 15: Main Decoder Internal Structure

The logical approach of the Main Decoder is to first split the opcode[6 : 0] into a 7-bit binary number, determine the instruction type through AND logic, and then generate the output status signals via OR logic.

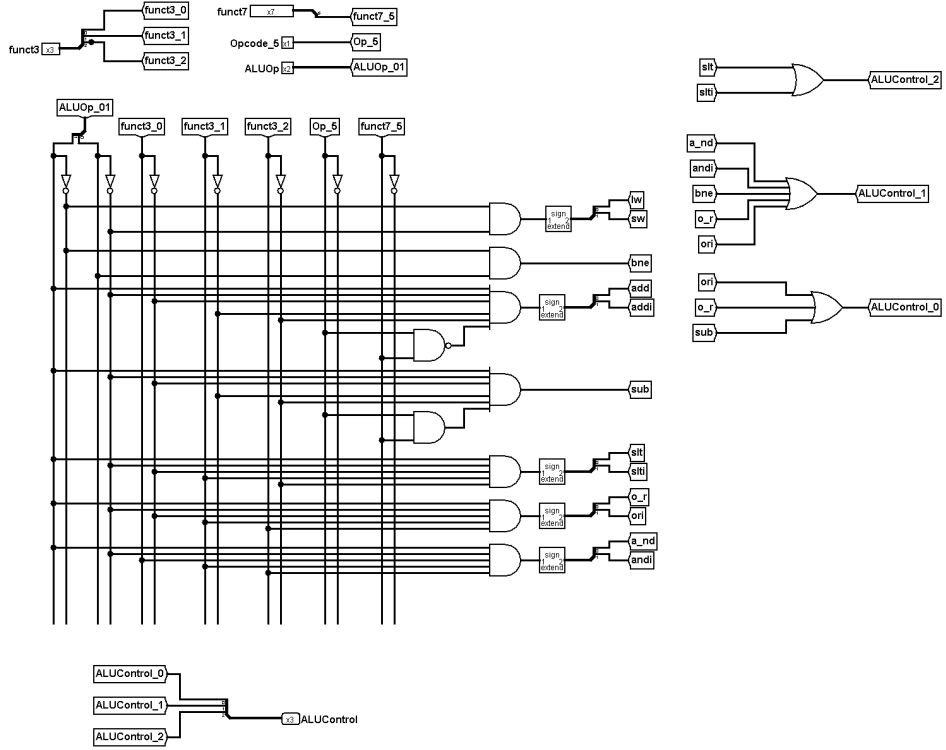


Figure 16: ALU Decoder Internal Structure

The logical approach of the ALU Decoder is to first split funct3 and funct7 into a 3-bit and a 7-bit binary number, respectively. It then determines the exact instruction name by performing AND logic operations on opcode[5], funct7[5], and the ALUOp signal output by the Main Decoder. Finally, it generates the output status signals via OR logic operations.

3.2 Data Path Explanation

The data path controls the flow trajectory of data among various modules from the fetch to the execution of each instruction. Controlling the data flow direction according to different instructions is the key to instruction implementation, which is achieved by using Multiplexers (MUX) and their selection signals.

We use 3 MUXes in data path design:

- **MUX for Next PC selection (PCSrc):** This MUX selects between the next sequential PC address ($PC + 4$) and the branch target address (calculated by adding ImmExt to the current PC) as the next PC value. For non-branch instructions, it selects $PC + 4$; for bne instruction, if the Zero flag is not set, it selects the branch target address.

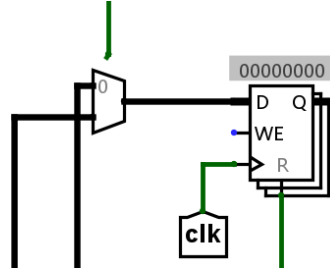


Figure 17: MUX for Next PC selection (PCSrc)

PCNext has two input options: $pc + 4$ and PCTarget. The selection is controlled by the PCSrc signal. For non-branch instructions, PCSrc is $1'b0$, selecting $pc + 4$ as the next PC value. For the jal instruction, PCSrc is $1'b1$, selecting PCTarget as the next PC value. For the bne instruction, if the Zero flag is not set, PCSrc is $1'b1$, selecting PCTarget as the next PC value.

- **MUX for ALU second operand (ALUSrcB):** This MUX selects between the second read data from the Register File (RD2) and the extended immediate value (ImmExt) as the second operand for the ALU. For R-type instructions, it selects RD2; for I1-type and S-type instructions, it selects ImmExt.

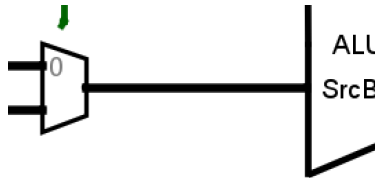


Figure 18: MUX for ALU second operand (ALUSrcB)

ALUSrcB has two input options: RD2 and ImmExt. The selection is controlled by the ALUSrcB signal. For R-type instructions, ALUSrcB is 1'b0, selecting RD2 as the second operand for the ALU. For I1-type and S-type instructions, ALUSrcB is 1'b1, selecting ImmExt as the second operand for the ALU.

- **MUX for Result selection (ResultSrc):** This MUX selects between the ALU result (ALUResult) and the data read from Data Memory (DMReadData) as the data to be written back to the Register File. For R-type, I1-type, and J-type instructions, it selects ALUResult; for lw instruction, it selects DMReadData.

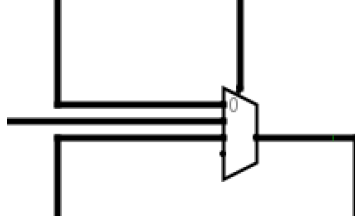


Figure 19: MUX for Result selection (ResultSrc)

ResultSrc has three input optional: ALUResult, ReadData, and PC +4. The selection is controlled by the ResultSrc signal. For R-type, I1-type, and S-type instructions, ResultSrc is 2'b00, selecting ALUResult as the data to be written back to the Register File. For the lw instruction, ResultSrc is 2'b01, selecting ReadData as the data to be written back to the Register File. For the jal instruction, ResultSrc is 2'b10, selecting PC +4 as the data to be written back to the Register File.

In summary, we present the respective control signals corresponding to each instruction, which also serves as an important basis for the CTRL connection.

Instruction	PCSrc	MemWrite	ALUControl	ALUSrcB	ImmSrc	RegWrite	ResultSrc
I1/Load	1'b0	1'b0	3'b000	1'b1	2'b00	1'b1	2'b01
S/Store	1'b0	1'b1	3'b000	1'b1	2'b01	1'b0	–
R/OP	1'b0	1'b0	Varies	1'b0	–	1'b1	2'b00
I1/OP-Imm	1'b0	1'b0	Varies	1'b1	2'b00	1'b1	2'b00
B/Branch	Varies	1'b0	3'b010	1'b0	2'b10	1'b0	–
J/JAL	1'b1	1'b0	—	-	2'b11	1'b1	2'b10

Table 7: Control Signals for Each Instruction Type

4 Assembler Building

```

1 INSTRUCTIONS_SET = {
2     "add": {"opcode": "0110011", "funct3": "000", "funct7": "0000000",
3     "type": "R"},
4     "sub": {"opcode": "0110011", "funct3": "000", "funct7": "0100000",
5     "type": "R"},

```

```

4     "or": {"opcode": "0110011", "funct3": "110", "funct7": "0000000", "
type": "R"},
5     "and": {"opcode": "0110011", "funct3": "111", "funct7": "0000000",
"type": "R"},
6     "slt": {"opcode": "0110011", "funct3": "010", "funct7": "0000000",
"type": "R"},
7     "slti": {"opcode": "0010011", "funct3": "010", "type": "I1"},
8     "lw": {"opcode": "0000011", "funct3": "010", "type": "I1"},
9     "addi": {"opcode": "0010011", "funct3": "000", "type": "I1"},
10    "ori": {"opcode": "0010011", "funct3": "110", "type": "I1"},
11    "andi": {"opcode": "0010011", "funct3": "111", "type": "I1"},
12    "sw": {"opcode": "0100011", "funct3": "010", "type": "S"},
13    "bne": {"opcode": "1100011", "funct3": "001", "type": "B"},
14    "jal": {"opcode": "1101111", "type": "J"},
15 }

```

Listing 1: The Instructions Set

```

1 import argparse
2 from typing import List, Tuple, Optional
3
4 import instructions
5
6
7 def preprocess_instruction(line: str) -> Tuple[str, List[str]]:
8     """
9     Turn an assembly line into (mnemonic, operands []).
10
11     Supports:
12     - commas: addi x1, x0, 1
13     - load/store addressing: lw x1, 0(x2) -> ["x1","0","x2"]
14     """
15     parts = line.replace(",", "").replace("(", " ").replace(")", "").
split()
16     if not parts:
17         raise ValueError("Empty instruction")
18     mnemonic = parts[0]
19     operands = parts[1:]
20     return mnemonic, operands
21
22
23 def register_to_binary(register: str) -> str:
24     if not register.startswith("x"):
25         raise ValueError(f"Invalid register name: {register}")
26     try:
27         reg_num = int(register[1:])
28     except Exception:
29         raise ValueError(f"Invalid register name: {register}")
30     if not (0 <= reg_num <= 31):
31         raise ValueError(f"Register out of range x0-x31: {register}")
32     return format(reg_num, "05b")
33
34

```

```

35 def immediate_to_binary(immediate: str, bits: int) -> str: #please
    learn: can not read this yet
36     imm = int(immediate, 0) # allow decimal and 0x.. forms
37     if imm < 0:
38         imm = (1 << bits) + imm
39     return format(imm & ((1 << bits) - 1), f"0{bits}b")
40
41
42 def bin_to_hex32(bin_str: str) -> str:
43     """32-bit binary string -> 8-hex-digit string (no 0x prefix)."""
44     if len(bin_str) != 32:
45         raise ValueError(f"Expected 32-bit machine code, got {len(
46 bin_str)} bits")
47     return format(int(bin_str, 2), "08x")
48
49 def assemble_to_machine_code(assembly_instruction: str) -> dict:
50     mnemonic, operand = preprocess_instruction(assembly_instruction)
51     inst = instructions.INSTRUCTIONS_SET.get(mnemonic)
52     if inst is None:
53         raise ValueError(f"Unknown mnemonic: {mnemonic}")
54
55     opcode = inst["opcode"]
56     inst_type = inst["type"]
57
58     if inst_type == "R":
59         # R-type: mnemonic rd, rs1, rs2
60         rd, rs1, rs2 = operand
61         machine_code = (
62             f"{inst['funct7']}"
63             f"{register_to_binary(rs2)}"
64             f"{register_to_binary(rs1)}"
65             f"{inst['funct3']}"
66             f"{register_to_binary(rd)}"
67             f"{opcode}"
68         )
69
70     elif inst_type == "I1":
71         # I-type: addi/andi/ori - rd, rs1, imm
72         # lw after preprocess is rd, imm, rs1
73         if mnemonic == "lw":
74             rd, imm, rs1 = operand
75         else:
76             rd, rs1, imm = operand
77         imm_binary = immediate_to_binary(imm, 12)
78         machine_code = (
79             f"{imm_binary}"
80             f"{register_to_binary(rs1)}"
81             f"{inst['funct3']}"
82             f"{register_to_binary(rd)}"
83             f"{opcode}"
84         )
85
86

```

```

87 elif inst_type == "S":
88     # S-type: sw rs2, imm(rs1) -> preprocess gives [rs2, imm, rs1]
89     rs2, imm, rs1 = operand
90     imm_binary = immediate_to_binary(imm, 12)
91     machine_code = (
92         f"{imm_binary[:7]}"
93         f"{register_to_binary(rs2)}"
94         f"{register_to_binary(rs1)}"
95         f"{inst['funct3']}"
96         f"{imm_binary[7:]}"
97         f"{opcode}"
98     )
99
100 elif inst_type == "B":
101     # B-type: bne rs1, rs2, imm
102     rs1, rs2, imm = operand
103     imm_binary = immediate_to_binary(imm, 13) # includes sign bit
104     # imm[12] imm[10:5] rs2 rs1 funct3 imm[4:1] imm[11] opcode
105     machine_code = (
106         f"{imm_binary[0]}"
107         f"{imm_binary[2:8]}"
108         f"{register_to_binary(rs2)}"
109         f"{register_to_binary(rs1)}"
110         f"{inst['funct3']}"
111         f"{imm_binary[8:12]}"
112         f"{imm_binary[1]}"
113         f"{opcode}"
114     )
115
116 elif inst_type == "J":
117     # J-type: jal rd, imm
118     rd, imm = operand
119     imm_binary = immediate_to_binary(imm, 21)
120     # imm[20] imm[10:1] imm[11] imm[19:12] rd opcode
121     machine_code = (
122         f"{imm_binary[0]}"
123         f"{imm_binary[10:20]}"
124         f"{imm_binary[9]}"
125         f"{imm_binary[1:9]}"
126         f"{register_to_binary(rd)}"
127         f"{opcode}"
128     )
129
130 else:
131     raise ValueError(f"Unknown instruction type: {inst_type}")
132
133     return {"Assembly": assembly_instruction, "MachineCode":
134     machine_code, "Type": inst_type}
135
136 def strip_comment(line: str) -> str:
137     # support '#' and ';' comments
138     for sep in ("#", ";"):
139         if sep in line:

```

```

140         line = line.split(sep, 1)[0]
141     return line.strip()
142
143
144 def assemble_file(
145     in_path: str,
146     out_path: Optional[str] = None,
147     emit_hex: bool = True,
148 ) -> List[str]:
149     """
150     Assemble a .s file line by line.
151     Returns output lines (also writes to out_path if provided).
152     """
153     output_lines: List[str] = []
154     pc = 0 # each instruction 4 bytes (RV32I)
155     with open(in_path, "r", encoding="utf-8") as f:
156         for lineno, raw in enumerate(f, start=1):
157             line = strip_comment(raw)
158             if not line:
159                 continue
160             try:
161                 res = assemble_to_machine_code(line)
162                 bin_code = res["MachineCode"]
163                 if emit_hex:
164                     hex_code = bin_to_hex32(bin_code)
165                     output_lines.append(f" {hex_code} ")
166                 else:
167                     output_lines.append(f" {bin_code} ")
168                 pc += 4
169             except Exception as e:
170                 output_lines.append(f"ERROR line {lineno}: {e}      # {
171 raw.rstrip()}" )
172                 # keep going
173
174     if out_path:
175         with open(out_path, "w", encoding="utf-8") as wf:
176             wf.write("\n".join(output_lines) + "\n")
177     return output_lines
178
179 def main():
180     parser = argparse.ArgumentParser(description="Simple RV32I
181 assembler (subset) - batch mode")
182     parser.add_argument("-f", "--file", help="Input assembly file (.s)"
183 )
184     parser.add_argument("-o", "--out", help="Output text file (optional
185 )" )
186     parser.add_argument("--bin", action="store_true", help="Emit binary
187 instead of hex")
188     args = parser.parse_args()
189
190     if args.file:
191         lines = assemble_file(args.file, out_path=args.out, emit_hex=
192 not args.bin)

```

```

188     print("\n".join(lines))
189     else:
190         # fallback: single-line interactive mode (original behavior)
191         assembly_instruction = input("Enter assembly instruction: ").
strip()
192         result = assemble_to_machine_code(assembly_instruction)
193         print("Assembly Instruction:", result["Assembly"])
194         print("Machine Code:", result["MachineCode"])
195         print("Instruction Type:", result["Type"])
196
197
198 if __name__ == "__main__":
199     main()

```

Listing 2: The Assembler Code

The Assembler converts RISC-V assembly instructions into 32-bit machine code. It first parses each instruction with `preprocess_instructions` to extract the mnemonic and operands (handling commas, parentheses, and load/store addressing). Then `assemble_to_machine_code` assembles the machine code based on the instruction type: it converts register names to 5-bit binary, converts immediates to the required bit width (using two's complement for negatives), and concatenates fields in the order specified by each instruction format. Finally, `assemble_file` processes assembly files line by line, converts each instruction, and outputs hex or binary while maintaining a PC that increments by 4 bytes per instruction.

5 Test Cases and Verification

5.1 Test Cases

We have designed a series of test cases, and judge whether the CPU executes instructions correctly by monitoring the states of the Register File and the Data Memory.

```

1 addi x1, x0, 10 # Initialize x1 with 10
2 addi x2, x0, -20 # Initialize x2 with -20
3 addi x3, x0, 5 # Initialize x3 with 5
4 add x4, x1, x2 # x4 = -10
5 add x5, x1, x3 # x5 = 15
6 sub x6, x1, x2 # x6 = 30
7 sub x7, x1, x3 # x7 = 5
8 and x8, x1, x2 # x8 = 8
9 and x9, x1, x3 # x9 = 0
10 or x10, x1, x2 # x10 = -18
11 or x11, x1, x3 # x11 = 15
12 slt x12, x1, x2 # x12 = 0
13 slt x13, x2, x1 # x13 = 1
14 slti x14, x1, -8 # x14 = 0
15 slti x15, x1, 28 # x15 = 1
16 addi x16, x16, 1 # x16 = 1
17 addi x17, x0, 3 # x17 = 3
18 bne x16, x17, -8 # branch to addi x16, x16, 1 if x16 != x17
19 jal x16, 8 # jump to addi x19, x1, 8
20 addi x17, x0, 7 # x17 = 7

```

```

21 andi x19, x1, 8 # x19 = 8
22 andi x20, x2, 18 # x20 = -2
23 ori x21, x1, 8 # x21 = 10
24 ori x22, x2, 18 # x22 = -2
25 sw x19, 0(x5) # Store x19 at address in x5
26 sw x19, 4(x5) # Store x19 at address in x5 + 4
27 lw x18, 0(x5) # Load from address in x5 to x18

```

Listing 3: Test Cases

5.2 Verification

Importing the `output.txt` file (generated by converting `testcase.s` via `Assembler_batch.py`) into `Convert.py` will yield a hexadecimal file compatible with Logisim-Evolution v3.0, which can be directly imported into the ROM for automated testing.

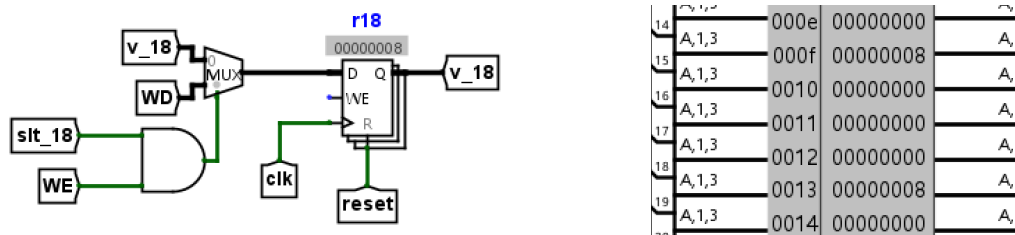


Figure 20: Verification Sign

Testing confirms that our CPU operates normally.

6 Summary of the project

This project involved designing and implementing a simple single-cycle RISC-V CPU that supports a subset of the RV32I instruction set. The CPU architecture includes essential components such as the Program Counter (PC), Instruction Memory, Register File, ALU, Data Memory, and Control Unit. Each module was carefully designed to ensure correct functionality and seamless integration within the CPU.

Students have initially learned and understood the RISC-V instruction set and the basic architecture of computers, laying a solid foundation for their subsequent studies.

7 Team Roles and Responsibilities

- **Mingxuan Li:** Responsible for assembler development and test case design.
- **Ruiyang Xu:** Responsible for the design and implementation of key modules and the writing of the final report.
- **Kaiwen Yang:** Responsible for datapath implementation and test case design, CPU testing

8 Acknowledgements

We would like to express our sincere gratitude to our course teaching assistants for their invaluable guidance and support throughout this project. Their expertise and encouragement have been instrumental in helping us navigate the complexities of CPU design and implementation.